Expert Systems With Applications 185 (2021) 115501

Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa

A hybrid matheuristic for the Two-Stage Capacitated Facility Location problem

Gabriel Souto [a,1], Igor Morais [a,*,2], Geraldo Regis Mauri [b,3], Glaydston Mattos Ribeiro [c,4], Pedro Henrique González [a,5]

[a] DEPIN/PPCIC – Federal Center for Technological Education of Rio de Janeiro, Brazil
[b] DCOMP – Federal University of Espírito Santo, Brazil
[c] PET – Federal University of Rio de Janeiro, Brazil

ARTICLE INFO

Keywords:
Two-Stage Capacitated Facility Location
Clustering Search
ALNS
Local Branching
Matheuristic

ABSTRACT

In the class of supply chain problems, the Two-Stage Capacitated Facility Location (TSCFL) is defined by optimal locations for installing factories and warehouses to meet the demand of customers. The problem aims to minimize operating costs: opening facilities and the flow of products from factories to customers, passing through warehouses, meeting the capacity constraints of factories and warehouses and customers' demand. To solve this problem, a hybridization of Clustering Search (CS), Adaptive Large Neighborhood Search (ALNS) and Local Branching (LB) is proposed. This hybridization is a new and interesting approach which has found high quality solutions in low computational time. To show that, computational experiments were performed using benchmark instances. The results showed that the proposed method outperforms the current state-of-art for the TSCFL for 40 out of 50 instances.

1. Introduction

Success of any company is related to its operational efficiency, which is closely linked to the geographical layout of its facilities (Zhong, Xu, Klotz, & Newman, 2017). A good location can provide a reduction in transportation costs, increasing the company's profits and meeting customers' demand efficiently.

The facility location problem proposes to define an optimal system for the distribution of facilities (factories, warehouses, points of sale, etc.) to meet customers' demand at the lowest possible cost. An extension of this problem is the Two-stage Capacitated Facility Location (TSCFL), which considers the capacity restrictions of factories and warehouses, and a level interposed between factories and customers (warehouses).

The products must flow from the factories through the warehouses until reaches the customers (factory →warehouse →customer). The final cost is defined by the cost of opening the facilities plus the cost to transport from factories to warehouses and from warehouses to customers. Therefore, minimizing this total cost is the objective of the problem.

Variants of the TSCFL problem were proposed during the years. They concern either a single-source or multi-source with a single commodity or multi-commodity planning. Many combinations of this characteristics were studied in the literature. The single-source with multi-commodity was proposed by Geoffrion and Graves (1974), which used a Benders decomposition for solving it. Another variant, proposed by Tragantalerngsak, Holt, and Ro"nnqvist (1997), considers that the warehouses are supplied by only one factory. Pirkul and Jayaraman (1998) considered a formulation with multiple sources and multi-commodity and proposed a Lagrangian relaxation for solving it. Later, Mauri et al. (2021) proposed two metaheuristics for the same problem. Understanding that, from a practical point of view, not only the warehouse

* Corresponding author.
E-mail addresses: gabriel.augusto@aluno.cefet-rj.br (G. Souto), igor.morais@eic.cefet-rj.br (I. Morais), geraldo.mauri@ufes.br (G.R. Mauri), glaydston@pet.coppe.ufrj.br (G.M. Ribeiro), pegonzalez@eic.cefet-rj.br (P.H. González).
[1] ORCID ID: https://orcid.org/0000-0003-0491-9316
[2] ORCID ID: https://orcid.org/0000-0003-2006-5250
[3] ORCID ID: https://orcid.org/0000-0002-8393-7741
[4] ORCID ID: https://orcid.org/0000-0001-8452-057X
[5] ORCID ID: https://orcid.org/0000-0003-0057-7670

https://doi.org/10.1016/j.eswa.2021.115501
Received 19 January 2021; Received in revised form 28 April 2021; Accepted 25 June 2021
Available online 5 July 2021
0957-4174/© 2021 Elsevier Ltd. All rights reserved.

location is important, but also its size, Wu, Yang, Chu, and Zhou (2015) proposed a mathematical formulation that expands the original one and also considers the warehouse size. In addition, to deal with this variant, Wu et al. (2015) proposed a Lagrangian relaxation technique. Another variant which considers that the customers can only be attended by one warehouse is presented in Yang, Chen, Chu, and Wang (2019).

Another approaches of the TSCFL can be found in the literature, as seen in Klose (2000), Tragantalerngsak, Holt, and Ronnqvist (2000) and Litvinchev and Ozuna (2012). Klose (2000) applied a Lagrangian relaxation based on *cut-and-relax* as an alternative for solving the problem, considering that the factories are fixed, which means that they are previously allocated. Fixed factories were also considered in Tragantalerngsak et al. (2000), which presented an exact method for solving the problem. Litvinchev and Ozuna (2012) has already proposed the use of Lagrangian relaxation to solve the problem considering the location of both factories and warehouses. More details on some of the different approaches to TSCFL can be found at Klose and Drexl (2005).

In this work, the same variant presented by Litvinchev and Ozuna (2012) is studied. This variant consists in determining the location of both warehouses and factories. This same approach was considered in the most recent works on TSCFL, described briefly below.

In Fernandes et al. (2014), a Genetic Algorithm (GA) was presented to solve the TSCFL. The authors proposed a set of instances with different characteristics to evaluate the proposed GA. This same set of instances was used in Louzada, Mauri, and Ribeiro (2016), where a hybrid method was proposed based on the combination of the Clustering Search (CS) method, to define the factories and warehouses to be installed, with an exact algorithm, used to define the flow of products between factories, warehouses and customers. The CS was able to find better solutions compared to the AG in lower computational times. A hybrid evolutionary algorithm with machine learning (HEA/FA) was presented in Guo, Cheng, and Wang (2017). The results obtained were competitive with the ones obtained with AG and CS described above. In Biajoli, Chaves, and Lorena (2019) a *Biased Random-Key Genetic Algorithm* (BRKGA) for the TSCFL was proposed. The results obtained were competitive with those presented so far and redefined the state-of-art for the TSCFL. Finally, in Gonzalez et al. (2019), a hybrid method was proposed based on the *Greedy Randomized Adaptive Search Procedure* (GRASP) with a *Local Branching* procedure. The GRASP, presented in Gonzalez et al. (2019), obtained relevant results and was able to keep the computational time stable in relation to the increase in the instances' size.

To solve this variant of the TSCFL, we propose a hybridization of Clustering Search (CS) and Adaptive Large Neighborhood Search (ALNS) metaheuristics with the Local Branching technique to find solutions for the TSCFL. As far as we know, this methodology has not yet been addressed in the literature for any problem. ALNS was proposed by Ropke and Pisinger (2006) and uses processes of destruction and reconstruction adaptively to improve the quality of the solution, while the CS (Oliveira, Chaves, & Lorena, 2013) metaheuristic avoids excessive calls on local search procedures, which can reduce the computational time. After that, a Local Branching (Fischetti & Lodi, 2003) procedure is called, using the best solution found as base solution, in order to try to improve it. Matheuristics using ALNS were already used in other applications (Keskin & Çatay, 2018) and showed the effectiveness of ALNS to diversify the search space.

The remainder of this paper is organized as follows: a detailed description of TSCFL and a mixed integer programming formulation is presented in Section 2, followed by the description of the proposed method in Section 3. Computational experiments are described in Section 4. Finally, the conclusions and future works are presented in Section 5.

2. Problem description and mathematical formulation

Given a set of factories and warehouses to be opened, the flow of products from factories to warehouses (level 1) and the flow from warehouses to customers (level 2), is a typical scenario modeled by the Two-Stage Capacitated Facility Location (TSCFL). Also the TSCFL aims to make this logistic plan, while minimizing the total cost. Having said that, the TSCFL comes to facilitate the ambitions of reducing costs, saving resources and increasing profits, which is the objective of companies in general.

The main objective of the problem is to minimize costs, as long as the capacity constraints of factories and warehouses are met, and customers' demands are also satisfied. The costs mentioned above can be separated into opening costs and transportation costs. The opening costs are the cost to open (build/install) a factory or warehouse, while the transportation cost consists of the cost to transport products from factories to warehouses, and from warehouses to customers.

As can be seen in Fig. 1, $\mathscr{I}$ represents the factories, $\mathscr{J}$ the warehouses, and $\mathscr{K}$ the set of all customers. For each factory $i \in \mathscr{I}$, its capacity b_i, its opening cost f_i and the costs c_{ij} of shipping products to all warehouses $j \in \mathscr{J}$ are defined. Similarly, for each warehouse $j \in \mathscr{J}$, its capacity p_j, its opening cost g_j and the costs d_{jk} of sending products to all customers $k \in \mathscr{K}$ are defined. For each customer $k \in \mathscr{K}$, q_k is defined as the demand to be satisfied.

Considering the definitions above, the TSCFL can be defined as determining a subset of factories $\overline{\mathscr{I}} \subseteq \mathscr{I}$ and warehouses $\overline{\mathscr{J}} \subseteq \mathscr{J}$ so that customer demands are met and the sum of opening and transportation costs is minimal.

In order to represent the TSCFL as Mixed Integer Programming (MIP) problem the decisions to be made at each step have to be defined in term of decision variables. Given that, let us define $y_i, i \in \mathscr{I}$ as decision variables that indicate whether the factory i will be opened or not. Also, let $z_j, j \in \mathscr{J}$ be defined as decision variables that indicate whether the

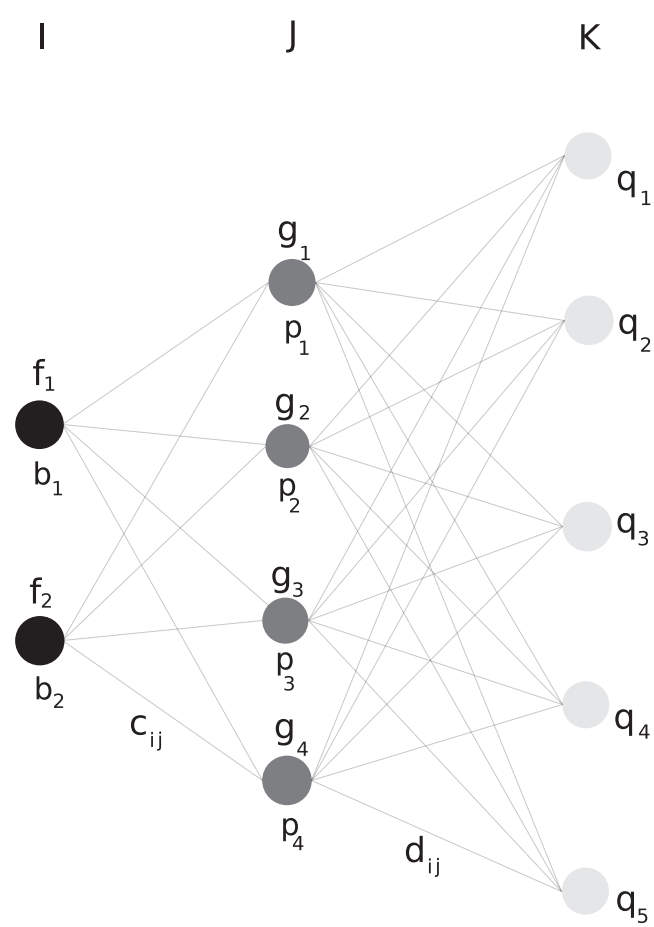

Fig. 1. Network with 2 factories, 4 warehouses and 5 customers.

warehouse $j$will be opened or not. At last, the decision variables $x_{ij}, i \in \mathscr{I}, j \in \mathscr{J}$, and $s_{jk}, j \in \mathscr{J}, k \in \mathscr{K}$, indicate how much flow is being sent from the factory i to the warehouse $j$and from the warehouse $j$to the customer k, respectively. Now, the TSCFL can be defined as a MIP problem as (Fernandes et al., 2014):

$$\begin{aligned} in \quad & \sum_{i\in\mathscr{I}} f_i y_i + \sum_{j\in\mathscr{J}} g_j z_j + \sum_{i\in\mathscr{I}}\sum_{j\in\mathscr{J}} c_{ij}x_{ij} + \sum_{j\in\mathscr{J}}\sum_{k\in\mathscr{K}} d_{jk}s_{jk} \\ \text{s.t}: \quad & \sum_{j\in\mathscr{J}} s_{jk} = q_k \quad \forall k \in \mathscr{K} \end{aligned} \tag{1}$$

$$\sum_{i\in\mathscr{I}} x_{ij} = \sum_{k\in\mathscr{K}} s_{jk} \quad \forall j \in \mathscr{J} \tag{2}$$

$$\sum_{j\in\mathscr{J}} x_{ij} \leqslant b_i y_i \quad \forall i \in \mathscr{I} \tag{3}$$

$$\sum_{k\in\mathscr{K}} s_{jk} \leqslant p_j z_j \quad \forall j \in \mathscr{J} \tag{4}$$

$$x_{ij} \in \mathbb{R}^+ \quad \forall i \in \mathscr{I}, j \in \mathscr{J} \tag{5}$$

$$s_{jk} \in \mathbb{R}^+ \quad \forall j \in \mathscr{J}, k \in \mathscr{K} \tag{6}$$

$$y_i \in \{0,1\} \quad \forall i \in \mathscr{I} \tag{7}$$

$$z_j \in \{0,1\} \quad \forall j \in \mathscr{J} \tag{8}$$

Constraints (1) ensure that each customer has their demand satisfied. Constraints (2) guarantee that the sum of the quantity of products that arrive in a warehouse needs to be greater than or equal to the quantity that this warehouse sends to customers. Also, Constraints (3) and (4) relate the flow sent by a factory (or warehouse) and whether they are open or not. Finally, Constraints (5) define the domain of each of the decision variables.

3. Solution method

This section presents the main contribution of this paper, which is a hybridization of Clustering Search – CS (Oliveira et al., 2013), Adaptive Large Neighborhood Search – ALNS (Ropke & Pisinger, 2006) and Local Branch – LB (Fischetti & Lodi, 2003), here called CS-ALNS-LB. The remainder of this section is organized as follows: First, constructive, destructive and repairing heuristic components are presented in Subsection 3.1, followed by a local search procedure described in Subsection 3.2. In Subsection 3.3, the ALNS metaheuristic is presented. After that, in Subsection 3.4, a Local Branching for the TSCFL is presented. At last, in Subsection 3.5, the hybrid CS-ALNS-LB matheuristic is presented, uniting all components described in Section 3.

It is important to remark that a vital part of every metaheuristic is to calculate the value of a given solution's objective function. Although the opening costs are trivial to calculate, to calculate the transportation costs, a Minimum Cost Flow problem (Ahuja, Magnanti, & Orlin, 1988) has to be solved. From this point on, whenever necessary to calculate the transportation cost, except on Local Branching, we must consider the use of the *Network Simplex* algorithm of the CPLEX commercial package.

3.1. Semi-greedy constructive heuristic

The constructive heuristic proposed here to obtain a feasible solution for the TSCFL consists of: Choosing a subset of factories to be opened; Choosing a subset of warehouses to be opened; Calculating the minimum cost flow.

The selection of a subset of facilities to be opened is carried out as follows: there is a repetition loop that chooses a factory randomly from a restricted candidates list, here called CL_1, until the sum of the capacities of the factories is greater than or equal to the sum of customers' demands. To build CL_1, the following equation was used as a cost-benefit ratio:

$$h_i^1 = \frac{f_i + \sum_{j\in\mathscr{J}} c_{ij}}{b_i}, \ \forall i \in \mathscr{I} \tag{9}$$

Once the cost-benefit ratio is defined, it is possible to build CL_1, according to the following rule:

$$CL_1 = \left\{ i \in \mathscr{I} \setminus \mathscr{F} | h_{min}^1 \leqslant h_i^1 \leqslant h_{max}^1 + \alpha\left(h_{min}^1 - h_{max}^1\right) \right\}, \tag{10}$$

where $h_{min}^1 = \underset{i\in\mathscr{I}}{\operatorname{argmin}}\left\{h_i^1\right\}, h_{max}^1 = \underset{i\in\mathscr{I}}{\operatorname{argmax}}\left\{h_i^1\right\}, \alpha \in [0,1]$ and $\mathscr{F}$ it is the set of factories already open.

Similarly, for Step 2, a repetition loop is created in which a warehouse is chosen at each iteration until the sum of warehouse capacities is greater than or equal to the sum of customers' demands. Here, a restricted candidate list CL_2 is used considering the following cost-benefit ratio:

$$h_j^2 = \frac{g_j + \sum_{k\in\mathscr{K}} d_{jk}}{p_j}, \forall j \in \mathscr{J} \tag{11}$$

Using the cost-benefit ratio above, CL_2 is formed using the following rule:

$$CL_2 = \left\{ j \in \mathscr{J} \setminus \mathscr{D} | h_{min}^2 \leqslant h_j^2 \leqslant h_{max}^2 + \alpha\left(h_{min}^2 - h_{max}^2\right) \right\} \tag{12}$$

where $h_{min}^2 = \underset{j\in\mathscr{J}}{\operatorname{argmin}}\left\{h_j^2\right\}, h_{max}^2 = \underset{j\in\mathscr{J}}{\operatorname{argmax}}\left\{h_j^2\right\}, \alpha \in [0,1]$ and $\mathscr{D}$ is the set of warehouses already open. Parameter α in Eqs. (10) and (12) defines how greedy or random the constructive approach is.

In the last step, given the open facilities, the flow is obtained and the transportation and the opening costs are calculated, which together will be the value of the objective function of the problem. This sequence of steps is presented in detail in Algorithm 1:

Algorithm 1. Semi-greedy constructive – SGC

```
Data: I, J, K, α
begin
    F ← ∅;
    while Σ_{i∈F} b_i < Σ_{k∈K} q_k do
        CL_1^min ← GenerateCandidateList(I, F, α) ;
        F ← F ∪ rand(CL_1^min);
    end
    D ← ∅;
    while Σ_{j∈D} p_j < Σ_{k∈K} q_k do
        CL_2^min ← GenerateCandidateList(J, D, α) ;
        D ← D ∪ rand(CL_2^min);
    end
    s_best ← CalculateFlow(F, D);
end
```

The semi-greedy constructive heuristic feasible solution is used to initialize the CS-ALNS-LB.

3.2. Local search

Local searches are defined as a process of finding local minimums in a given neighborhood, using as input a previously defined solution. For this problem, three neighborhoods were defined: Exchanging an open facility for a closed facility (Exchange Neighborhood); Closing opened facilities (Close Neighborhood); and Opening closed facilities (Open Neighborhood). In all these neighborhoods, the local search procedure used a best-improvement strategy.

To better understand the proposed neighborhoods, consider that each solution can be represented by a union of two disjoint sets, one

containing the opened facilities ($\mathscr{O}$) and the other the closed ones ($\mathscr{C}$). Figs. 2–4 presents each neighborhood and its characteristics. Also, in Figs. 2–4, the dashed lines represent closed facilities ($\mathscr{C}$), while the solid lines represent the opened ones ($\mathscr{O}$).

The Exchange Neighborhood, represented in Fig. 2, starts by evaluating the exchange between each element of $\mathscr{O}$ and each element of $\mathscr{C}$. After evaluating each possible exchange, the pair of facilities $(\mathfrak{f}_i, \bar{\mathfrak{f}}_i)$, such that $\mathfrak{f}_i \in \mathscr{O}$ and $\bar{\mathfrak{f}}_i \in \mathscr{C}$ which leads to the best improvement of the inputted solution is selected.

The Close Neighborhood, represented in Fig. 3, consists in evaluate the removal of each element from $\mathscr{O}$ and adding it to $\mathscr{C}$. After evaluating the removal of each element from $\mathscr{O}$, the removal that leads to the best improvement of the inputted solution is selected.

The last neighborhood is the Open Neighborhood, represented in Fig. 4. It consists in evaluate the removal of each element from $\mathscr{C}$ and adding it to $\mathscr{O}$. After evaluating the addition of each element from $\mathscr{C}$ into $\mathscr{O}$, the addition, which leads to the best improvement of the inputted solution, is selected.

To reduce the number of explored solutions, a strategy defined in Louzada et al. (2016) was applied. This strategy consists of determining the maximum (β) and the minimum (π) number of factories and warehouses needed to guarantee that all customers' demands can be fulfilled. Having that, one may avoid exploring solutions with less than the minimum number of required facilities and solutions with more than the maximum number of required facilities. Avoid exploring these solutions is very important since evaluating a solution of the TSCFL is computationally expensive.

In order to determine β, for each type of facility, the following procedure was executed. First, the facilities are sorted in increasing order of capacities. After that, to define the maximum number needed of the chosen type of facility, a subset of facilities is created. This subset is created by adding facilities, from the facility with smaller capacity to the one with the maximum capacity, until the summation of capacities in this subset becomes greater or equal to the summation of customers' demands. Once this process is done, the obtained subset's cardinality is the maximum number needed for the chosen type of facility. The process to obtain π is similar to the process for determining β. The difference is that one needs to create the subset starting from the facility with maximum capacity and keep going in decreasing order until the summation of capacities in this subset becomes greater or equal to the summation of customers' demands.

This strategy is efficient for both Close and Open neighborhoods, and so it was employed on both of them.

All the presented neighborhoods were organized in a Random Variable Neighborhood Search (RVND) framework (Penna, Subramanian, & Ochi, 2013) and it is described in Algorithm 2:

Algorithm 2. RVND

```
Data: Solution s
begin
    N ← Random(Neighborhood N^swap, N^close, N^open);
    k ← 1;
    s_best ← s;
    while k ≤ 3 do
        Find best neighbor s' ∈ N^k(s);
        if f(s') < f(s_best) then
            s_best ← s';
            k ← 1;
        else
            k ← k + 1;
        end
    end
    return s_best
end
```

Fig. 2. Exchange – Open facility 1 and close facility 2.

Fig. 3. Remove – Close facility 6.

Fig. 4. Add – Open facility 3.

The RVND framework was chosen because it is difficult to determine and define the best order for exploring the neighborhoods. In Algorithm 2, N is defined as a vector of neighborhoods and N^k is the k-th position in this shuffled vector. The neighborhood order is not just problem related but also instance-related, so that a random order may be more effective than a fixed order. Given that, each time the RVND procedure starts, a new order is randomly generated. The RVND procedure is used to exploit the solution space in the CS-ALNS-LB.

3.3. Adaptive Large Neighborhood Search – ALNS

The Adaptive Large Neighborhood Search (ALNS) was proposed by Ropke and Pisinger (2006) and combines a constructive heuristic with several destructive and repair procedures, in an adaptive way, to explore different regions of the solution space. An important feature of the ALNS is that the repair procedures, the destructive procedures, or both the repair and destructive procedures have their selection based on its success. Considering that the ALNS components have to be built specifically for each problem, the destructive and repair processes for the TSCFL are defined in the next subsections.

3.3.1. Random destructive

The Random Destructive works as follows: Given a percentage for destruction, factories and warehouses are randomly closed. Regardless of the cost of the solution, facilities are closed without distinction. As a result, the Random Destructive is an important component to generate diversification.

3.3.2. Semi-Greedy Destructive

The Semi-Greedy Destructive uses multiple cost-benefit ratios and the RCL concept of the Semi-greedy Constructive Heuristic, presented in Section 3.1. In addition to the cost-benefit ratio defined in Section 3.1, two other cost-benefit ratios were proposed.

The first one takes into consideration the average transportation costs from factories to warehouses, Eq. (13), and from warehouses to customers, Eq. (14).

$$h_3 = \frac{f_i + \frac{1}{|\mathscr{J}|}\sum_{j\in\mathscr{J}} c_{ij}}{b_i}, \ \forall i \in \mathscr{I} \tag{13}$$

$$h_4 = \frac{g_j + \frac{1}{|\mathscr{K}|}\sum_{j\in\mathscr{J}} d_{jk}}{b_i}, \ \forall j \in \mathscr{J} \tag{14}$$

The second cost-benefit ratio proposed for the Semi-greedy Destructive procedure considers the opening cost of the facilities divided by their respective capacities. Eqs. (15) and (16) formally describe this idea:

$$h_5 = \frac{f_i}{b_i},\ \forall i \in \mathscr{I} \tag{15}$$

$$h_6 = \frac{g_j}{p_j},\ \forall j \in \mathscr{J} \tag{16}$$

These three cost-benefit ratios were used in an attempt to seek diversity, since it is impossible to know whether or not one will be better than the other for each problem instance. This situation arises since different real scenarios lead to different instances, which makes it impossible to guarantee, a priori, if any cost (or which cost) dominates the others.

The Semi-greedy Destructive procedure randomly selects one of the cost-benefit ratios and creates a candidate list composed of the factories or warehouses identifiers, and its respective cost-benefit values. This list is restricted by Eq. (17):

$$CL^{max} = \left\{ i \in \mathscr{I} \setminus \mathscr{F} | h^1_{max} \geqslant h^1_i \geqslant h^1_{min} + \delta\left(h^1_{max} - h^1_{min}\right) \right\} \tag{17}$$

This function is similar to that used to restrict candidates in the Semi-greedy Constructive procedure. However, instead of minimizing, now the facilities with maximum cost-benefit are desired. The idea is to choose candidates with high opening costs, but not necessarily the highest, to generate diversity.

Algorithm 3 presents the details of the Semi-greedy Destructive procedure. The parameters σ, δ and *random* are, respectively, the rate of destruction, the value that defines the size of the cost-benefit selection window, and whether the adaptive used is random or semi-greedy.

Algorithm 3. Semi-greedy Destrucitve – SGD

Data: $\mathcal{I}$,$\mathcal{J}$,$\mathcal{K}$, δ ,s_{temp},h_{random},σ, $random$
begin
  $s_{destroy} \leftarrow s_{temp}$;
  $max_{destroy} \leftarrow \sum_{i \in \mathcal{I}} s_{destroy} * \sigma$;
  $destroy \leftarrow 0$;
  while $destroy < max_{destroy}$ **do**
    $CL^{max}_{h_{random}} \leftarrow GenerateCandidateList(\mathcal{I}, \mathcal{F}, \delta, random)$;
    $s_{destroy} \leftarrow s_{destroy} \setminus rand(CL^{max}_{h_{random}})$;
    $destroy \leftarrow destroy + 1$;
  end
  $max_{destroy} \leftarrow \sum_{j \in \mathcal{J}} s_{destroy} * \sigma$;
  $destroy \leftarrow 0$;
  while $destroy < max_{destroy}$ **do**
    $CL^{max}_{h_{random}} \leftarrow GenerateCandidateList(\mathcal{J}, \mathcal{D}, \delta, random)$;
    $s_{destroy} \leftarrow s_{destroy} \setminus rand(CL^{max}_{h_{random}})$;
    $destroy \leftarrow destroy + 1$;
  end
  return $s_{destroy}$
end

When the algorithm receives the *random* $= 0$, the cost-benefit ratios are ignored, so facilities are randomly selected. Otherwise, when *random* $= 1$, the benefit-cost functions described above are used. Thus the *random* parameter is responsible for creating the dynamic between the random and semi-greedy. The Semi-greedy Destructive procedure may lead to infeasible solutions, which are repaired using the Semi-Greedy Repairer defined below.

3.3.3. Semi-Greedy Repairer

The Semi-Greedy Repairer is a procedure that receives an infeasible solution and returns a feasible one. In order to fulfill its role, a cost-benefit ratio is selected at random from the set of cost-benefit ratios defined in Section 3.3.2. Then, a Restricted Candidate List is generated using the same process of the Semi-greedy Constructive procedure. This process is presented in detail in Algorithm 4:

Algorithm 4. Semi-greedy Repairer – SGR

Data: $\mathcal{I}$,$\mathcal{J}$,$\mathcal{K}$, α_2 ,$s_{destroy}$,h_{random}
begin
  $s_{repair} \leftarrow s_{destroy}$;
  while $\sum_{i \in s_{repair}} b_i < \sum_{k \in \mathcal{K}} q_k$ **do**
    $CL_{h_{random}} \leftarrow GenerateCandidateList(\mathcal{I}, \mathcal{F}, \alpha_2)$;
    $s_{repair} \leftarrow s_{repair} \cup rand(CL_{h_{random}})$;
  end
  while $\sum_{j \in s_{repair}} p_j < \sum_{k \in \mathcal{K}} q_k$ **do**
    $CL_{h_{random}} \leftarrow GenerateCandidateList(\mathcal{J}, \mathcal{D}, \alpha_2)$;
    $s_{repair} \leftarrow s_{repair} \cup rand(CL_{h_{random}})$;
  end
  return s_{repair}
end

It is important to note that the indexes used to create the candidate lists are h_{random}, and this value is set before the Semi-greedy Repairer selects the cost-benefit ratio at random. Parameter α_2 defines how greedy or random the repairer approach is.

3.3.4. ALNS Structure for the TSCFL

The ALNS for the TSCFL combines the components described in the previous sections, using the following structure: First, a solution is built using SGC and then an iterative process of destruction (SGD) and repair (SGR) is repeated max_{iter} times. In the first iteration, the probability of choosing each of destructive strategy is the same. Every time the repair procedure finds a new global best solution, the probability of selecting the destructive used before the repairing is incremented. This probability is used as a mechanism to prioritize strategies that are behaving better for the current instance. This process is described in Algorithm 5:

Algorithm 5. Adaptive Large Neighborhood Search

Data: $\mathcal{I}, \mathcal{J}, \mathcal{K}, \alpha, \delta, \alpha2, \sigma, s, s_{best}$,$probability$, max_iter
begin
  $s \leftarrow SGC(\mathcal{I}, \mathcal{J}, \mathcal{K}, \alpha)$;
  $s_{best} \leftarrow s$;
  while $i < max_iter$ **do**
    $random \leftarrow RandomProbability(0, 1, probability)$;
    $h_{random} \leftarrow Random \in$ **card**(cost-benefit ratios) ;
    $s \leftarrow SGD(\mathcal{I}, \mathcal{J}, \mathcal{K}, \delta, s, h_{random}, \sigma, random)$;
    $s \leftarrow RSG(\mathcal{I}, \mathcal{J}, \mathcal{K}, \alpha2, s, h_{random}, random)$;
    UpdateProbability($s_{best}, s, probability, random$);
  end
  return s_{best};
end

The two functions, *RandomProbability* and the *UpdateProbability* are responsible for the adaptive procedure, the first one compares the probability with a random number between zero and one, if it's less than the probability it returns the Random Destructive, otherwise the Semi-greedy Destructive is returned. If a best solution is founded, the *UpdateProbability* increases in 0.1 the probability of choosing such a method, and decreases in 0.1 the probability of choosing another method.

3.4. Local Branching

The Local Branching (LB) technique, introduced by Fischetti and Lodi (2003), can be used in a way that mixed-integer programming models can be used to improve a given feasible solution. That said, one may say that LB uses a mixed-integer linear programming solver to

explore the solution space. This procedure can be correlated to a local search procedure, in which the neighborhoods are defined through the introduction of linear inequalities in the mixed-integer linear programming model. That strategy has already been used in the literature and provided good results (Gonzalez, Simonetti, Michelon, Martinhon, & Santos, 2016; Gonzalez & Brandão, 2018).

Specifically for the TSCFL, let us consider a solution $s = \langle \overline{y}, \overline{z} \rangle \in P$, where P is the polyhedron formed by the Constraints (1)–(8). The idea consists of adding the following LB constraints:

$$\sum_{i \in I | \overline{y}_i = 0} y_i + \sum_{i \in I | \overline{y}_i = 1} \left(1 - y_i\right) \leqslant \Delta_1, \tag{18}$$

$$\sum_{j \in J | \overline{z}_j = 0} z_j + \sum_{j \in J | \overline{z}_j = 1} \left(1 - z_j\right) \leqslant \Delta_2, \tag{19}$$

where Δ_1 and Δ_2 are a given positive integer, indicating the number of variables $y_i, i \in \mathscr{I}$ and $z_j, j \in \mathscr{J}$, which can have their value changed from one to zero and vice versa.

3.5. CS-ALNS-LB Matheuristic

The Clustering Search metaheuristic (CS) was proposed by Oliveira and Lorena (2007) as a generic way of combining metaheuristics and clustering to identify promising regions of the space to apply local search procedures. As calculating the cost function for this problem is computationally expensive, the CS hybrid metaheuristic is suited for this problem, because it only applies the local search procedure when several solutions in the same promising region are obtained.

As mentioned before in the literature (Oliveira & Lorena, 2007; Chaves, Lorena, Senne, & Resende, 2016; de Alvarenga Rosa, Machado, Ribeiro, & Mauri, 2016; González et al., 2019), the CS metaheuristic needs another technique to generate feasible solutions scattered in different regions of the solution. Any metaheuristic can be employed to generate solutions into CS. Oliveira et al. (2013) reports several successful approaches including Simulated Annealing (SA), Genetic Algorithm (GA) and GRASP. However, to the best of our knowledge, ALNS has not yet been used with CS for any problem. In addition, ALNS has shown good results for different problems outperforming other traditional metaheuristics such as SA and GA (Laporte, Musmano, & Vocaturo, 2010; Ribeiro, Laporte, & Mauri, 2012; Mauri, Ribeiro, Lorena, & Laporte, 2016; Kiefer, Hartl, & Schnell, 2017).

Having stated that, in this paper the ALNS presented in Section 3.3.4 was chosen to generate solutions into CS. The CS-ALNS works by, initially, separating each obtained solution in different clusters, until γ clusters are defined. After that, for each solution found we check in which cluster it should be placed, by using the Hamming distance to calculate the distance between the new solution and the cluster' *center*, to decide which cluster.

Once the cluster is decided, if the solution has a better cost than the center, it becomes the new center of the cluster, and the volume of the cluster is updated. Otherwise, just the volume of the cluster is updated. When that volume reaches a maximum value, max_{volume}, the local search process (described in Section 3.2) starts using the cluster center as incumbent solution. Another mechanism for escaping from local optimum is to verify whether a cluster fails to improve the best solution found after max_{inef} applications of the local search. Whenever that happens, the center of the cluster goes through a process of destruction and semi-greedy repair.

The CS-ALNS-LB combines the CS-ALNS adding a Local Branching phase (described in Section 3.4) to the best solution obtained by the CS-ALNS. The whole procedure is presented in Algorithm 6:

Algorithm 6. CS-ALNS-LB

```
Data: I, J, K, α, α2, δ, γ, max_volume, max_inef, σ
begin
    C_i ← 0, v_i ← 0, r_i ← 0 | i ∈ {1..γ};
    clt ← 0;
    s_best ← SGC(I, J, K, α);
    s ← s_best;
    probability ← 1/2 ;
    while i < max_iter do
        ALNS(I, J, K, α, δ, α2, σ, s, s_best, probability, 1);
        if clt < γ then
            C_i ← s, v_i ← 1, r_i ← 0, clt ← clt + 1;
        else
            i ← arg min_{i∈{1...γ}} H_i, v_i ← v_i + 1;
            UpdateCenter(C_i, s);
            if v_i = max_volume then
                C_i ← RVND(C_i), s ← Update(s_best, C_i);
                if C_i was modified then
                    r_i ← 0;
                else
                    r_i ← r_i + 1;
                end
            end
            if r_i = r_max then
                C_i ← SGD(I, J, K, δ, C_i, h_random, σ, random);
                C_i ← SGR(I, J, K, δ, C_i, h_random, σ, random);
            end
        end
    end
    s_best ← LocalBranching(s_best);
    return s_best;
end
```

The function UpdateCenter receives the new solution that is going to enter the cluster and the solution defined as the center of the cluster. Whenever this new solution has a better cost than the solution defined as cluster center, the new solution becomes the center of the cluster and the old center is discarded.

Table 1
Values tested and selected for each parameter of the CS-ALNS-LB.

Parameters	Intervals	Selected Values
α	$\{0.60; 0.65; 0.70; 0.75; 0.80; 0.85; 0.90;\}$	0.75
δ	$\{0.60; 0.65; 0.70; 0.75; 0.80; 0.85; 0.90;\}$	0.90
α_2	$\{0.60; 0.65; 0.70; 0.75; 0.80; 0.85; 0.90;\}$	0.90
γ	$(5, 20)$	19
max_{inef}	$(5, 20)$	18
max_{volume}	$(5, 15)$	10
max_{iter}	$[80, 400]$	260
σ	$\{0.1; 0.20; 0.30; 0.40\}$	0.40

4. Computational experiments

The proposed method (CS-ALNS-LB) was programmed in C++, using a GCC compiler version 7.5.0, and was executed ten times for each instance on an Intel Core i5 processor with a frequency of 2.30 GHz and 16 GB RAM.

The instances used were defined in Fernandes et al. (2014) and can be divided into two sets. The first set consists of 25 instances with 50 factories, 100 warehouses and 200 customers. In the second set, also with 25 instances, the size of the problem is doubled, which means 100 factories, 200 warehouses and 400 customers. Both sets are divided into five instance classes with different characteristics.

All instances can be find at https://github.com/pehgonzalez/OCA, along with the binary file to reproduce the experiments.

To determine the values for each parameter of the CS-ALNS-LB, the iRace (López-Ibáñez, Dubois-Lacoste, Cáceres, Birattari, & Stützle, 2016) package was used. As a fixed parameter for the LB, 50 s was used as a stopping criterion for the CPLEX solver. Table 1 presents the values tested for each parameter of the CS-ALNS-LB and their values used in the experiments.

The remainder of this section is organized into two subsections. The first one presents the computational results and discuss them. After that, a statistical analysis is done to verify whether the proposed approach is significantly different from the others in the literature.

4.1. Computational results

This section presents the results obtained by the proposed method and compares them to the results of the methods in the literature.

Tables 3 and 4 present the results obtained by CS-ALNS-LB, along with the results obtained by the other methods in the literature for the TSCFL. In both tables, the first columns indicate the class of the instance, the identifier and the best known lower bound (LowerB), presented by Guo et al. (2017). As for the CS+CPLEX method, the authors of Louzada et al. (2016) provided us with the code, which allowed us to rerun it in the same computational environment where the CS-ALNS-LB experiments were performed.

In order to do a fair comparison regarding the computational time between the proposed approach and the methods reported in the literature, a conversion factor was applied. Thus, the Single Thread Rating reported for each CPU, available at https://www.cpubenchmark.net/, was used. The corresponding factors are presented in Table 2.

Columns *AVG* and *BST* are calculated as distance from the Lower Bound presented by Guo et al. (2017), measured as $\frac{sol-LowerB}{LowerB} \times 100$, where *sol* indicates the *AVG* or *BST* solutions from each method. The column *Time* is calculated as the mean time in seconds for 10 executions.

Table 2
CPU conversion factor.

GA[a]	HEA/FA[b]	BRKGA+LS[c]	GRASPH[d]
0.46	1.04	0.83	1.18

[a] Fernandes et al. (2014) – Pentium Intel 2,3 GHz, 24 GB de RAM.
[b] Guo et al. (2017) – Intel Core i7 3.6 GHz, 8 GB de RAM.
[c] Biajoli et al. (2019) – Intel Core i5 1.7 GHz, 16 GB de RAM.
[d] Gonzalez et al. (2019) – Intel Core i7 4.2 GHz, 12 GB de RAM.

Figs. 5 and 6 present a summary of the results presented in Tables 3 and 4, to facilitate the analysis. In these figures, all the three dimensions evaluated in our computational experiments are considered, the AVG, BST and Time. The lighter color represents the AVG and the darker color represents the BST. Time is presented by the width of each section and is proportional to the computational time spent by each method. To better understand, consider that smaller the length, the better the solution is, while the smaller the width is, the faster is the method. With a simple look in Figs. 5 and 6, one may verify everything that was said during the discussion of the results.

Looking at Table 3, one may notice that the proposed method has the best mean of best solutions, the best mean of averages, the best time among all methods, and for four instances, it has new best solutions. The computational time is approximately 25% faster than the CS+CPLEX running on the same machine and almost 20% faster than the BRKGA. The method at least ties in 80% of the results presented, making the CS-ALNS-LB presented a viable option for the TSCFL. One may notice that our method obtains the minimum value in the best solutions for 24 out of the 25 instances.

In Class 3 instances, for which the mixed integer programming formulation failed to achieve optimum with the amount of available RAM used in Louzada et al. (2016), the CS-ALNS-LB obtained the best solutions in 4 out of 5 instances.

Table 4 shows that CS-ALNS-LB is still faster than the other methods, approximately two times, and the mean of the best solutions is better than all of the presented methods. Although the BRKGA-LS surpasses the mean of average solutions of CS-ALNS-LB by 0.02%, the CS-ALNS-LB outperformed the other methods. It is important to notice that the CS-ALNS-LB has achieved the best solutions for 16 out of 25 instances, and for two instances, a new minimum solution has been found. Another important remark is that BRKGA-LS uses a parallel approach to evaluate their population of solutions, with implicates that in a sequential version, BRKGA-LS will probably spend more computational time in order to achieve the presented results.

Analyzing Fig. 7, one may verify the gain of the proposed method in relation to the best solutions reported in the literature, represented by the gap between the dark and light blue. Given the presented data, it is possible to verify that in the worst case (BRKGA-LS) the improvement is 0.84%, while in the best case (GA) the improvement is 9.4%. From a practical point of view, improvements of 0.84% in their planning, as obtained by the proposed method, may implicate in a considerable improvement in the company's profit.

After analyzing the results, one may conclude that the proposed CS-ALNS-LB is a viable option for finding high-quality solutions in low computational time for TSCFL. In order to deepen the evaluation of the proposed method, the next section presents some statistical analysis to compare the CS-ALNS-LB with the two state-of-art methods.

4.2. Statistical analysis

In order to verify whether or not the differences of values obtained by the state-of-art strategies and the proposed method are statistically significant, a hypothesis test was executed. In addition to the tables of computational experiments for the proposed method, the hypothesis test was performed based on the results of the executions sent to us by the authors of Biajoli et al. (2019) and the new execution of the method CS+CPLEX in our machine. As it was not possible to prove that the data fit a normal distribution, the Willcoxon-Mann–Whitney (Hettmansperger & McKean, 1998) non-parametric test was used. This test can reject the null hypothesis of θ level, with the probability $(1 - \theta \times 100\%)$. In this paper we considered $\theta = 0.05$.

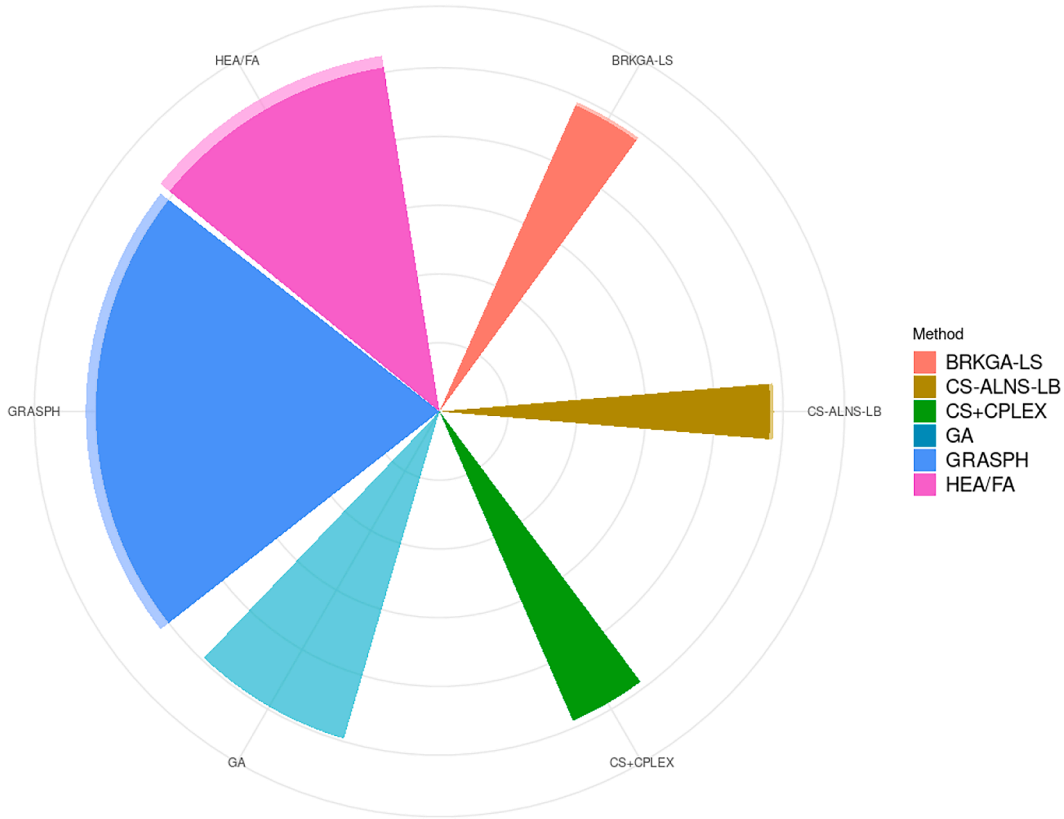

Fig. 5. Summary of the Results for Instances of 50 Factories.

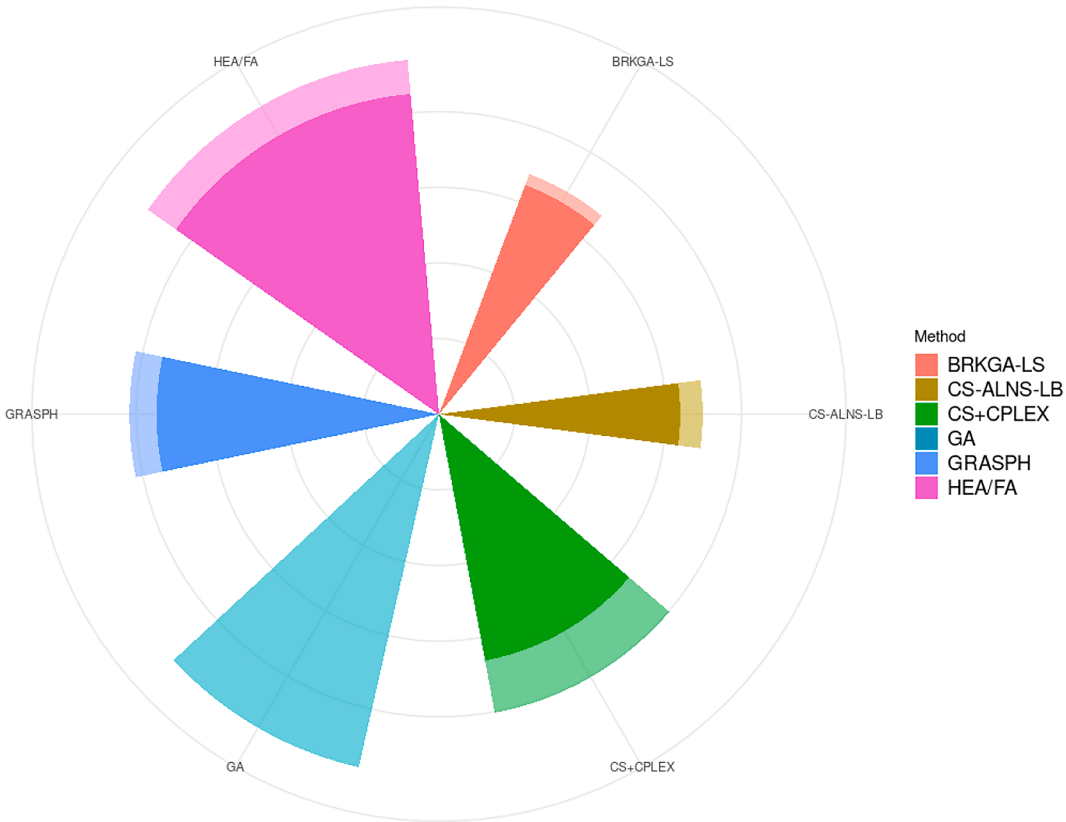

Fig. 6. Summary of the Results for Instances of 100 Factories.

Table 3
Results obtained for the first set of instances (50 factories, 100 warehouses and 200 customers).

Instances			GA[a]		CS+CPLEX			HEA/FA[b]			BRKGA+LS[c]			GRASPH[d]			CS-ALNS-LB		
Class	ID	LowerB	AVG	Time	BST	AVG	Time	BST	AVG	Time	BST	AVG	Time	BST	AVG	Time	BST	AVG	Time
1	1	721209.6	**0.13**	264.78	0.13	0.22	48.90	0.13	0.24	431.34	0.14	0.14	**19.32**	**0.13**	0.13	**4.50**	**0.13**	**0.13**	15.41
	2	730451.6	0.40	257.17	0.23	0.31	76.86	0.31	0.36	510.74	0.24	0.24	115.93	0.24	0.24	21.34	**0.23**	**0.23**	**21.02**
	3	731885.3	0.24	263.35	0.21	0.29	51.94	0.22	0.24	327.60	0.22	0.22	39.29	0.22	0.22	**30.46**	**0.21**	**0.21**	55.22
	4	721515	0.81	242.93	1.19	1.41	96.62	0.54	0.59	472.15	0.50	0.51	116.29	0.50	0.50	29.08	**0.50**	0.50	**18.98**
	5	713633.8	0.82	251.79	0.81	0.88	**56.14**	0.86	0.97	480.59	**0.81**	**0.81**	**59.10**	**0.81**	0.81	160.01	**0.81**	0.82	64.69
2	1	479860.2	**2.69**	144.39	2.68	3.27	27.69	2.74	2.82	239.25	2.69	2.69	16.84	2.69	2.69	383.27	**2.68**	**2.68**	**15.43**
	2	483072.2	2.30	144.16	2.30	2.62	81.36	2.34	2.41	206.37	2.30	2.31	60.58	2.30	2.34	368.58	**2.30**	**2.30**	64.89
	3	486018.5	2.14	150.60	1.86	2.03	**30.82**	1.87	1.89	173.38	1.87	1.87	117.20	1.88	1.94	590.94	**1.86**	**1.86**	48.02
	4	482374.6	2.04	142.25	2.01	2.01	83.02	2.07	2.12	168.04	2.02	2.05	59.59	2.02	2.02	365.57	**2.01**	**2.02**	**47.37**
	5	474803.3	3.14	126.08	3.12	3.39	36.98	**3.12**	3.12	176.79	**3.12**	**3.12**	**10.84**	**3.12**	3.12	590.87	**3.12**	**3.12**	33.33
3	1	2608800	3.07	125.90	3.07	3.07	**19.89**	3.14	3.30	121.36	3.11	3.11	126.94	3.22	3.30	596.03	**3.07**	**3.10**	104.13
	2	2616252	3.12	130.22	3.10	3.10	73.09	3.30	3.36	120.92	3.17	3.17	40.26	3.37	3.39	594.73	3.13	3.20	94.18
	3	2598277	3.11	123.56	3.09	3.10	**52.98**	3.30	3.39	167.70	**3.10**	**3.10**	72.94	3.23	3.32	591.33	**3.09**	3.14	77.21
	4	2612534	3.07	107.73	3.05	3.05	45.21	3.18	3.22	154.29	**3.10**	**3.10**	8.25	3.18	3.29	593.68	**3.05**	**3.10**	80.60
	5	2568856	**3.01**	110.36	3.01	3.01	47.45	3.17	3.19	163.91	3.13	3.13	40.23	3.14	3.22	593.27	**3.01**	3.03	76.79
4	1	525294.1	**3.14**	138.25	3.14	3.60	89.47	3.36	3.54	224.71	**3.14**	3.22	58.44	3.29	3.29	591.54	**3.14**	**3.14**	**39.42**
	2	526911.7	**2.33**	139.83	2.43	2.71	102.38	2.74	2.92	206.66	2.43	2.51	80.98	2.65	2.80	592.19	**2.43**	**2.45**	**57.23**
	3	532592.3	2.66	144.88	2.41	2.44	118.38	2.59	2.62	256.30	2.33	2.42	110.29	2.45	2.92	591.35	**2.30**	**2.44**	**67.49**
	4	529372	2.53	127.30	2.35	2.66	133.69	2.63	2.81	271.29	2.44	2.51	74.85	2.36	2.50	591.31	**2.35**	**2.36**	**55.50**
	5	521470.1	**3.13**	120.27	3.15	3.53	115.67	3.18	3.18	188.88	3.14	3.16	39.57	3.15	3.23	388.72	**3.12**	**3.12**	46.46
5	1	2743547	1.20	164.42	1.19	1.19	157.20	**1.16**	1.20	206.49	1.18	1.22	120.41	1.24	1.31	591.33	**1.16**	**1.18**	**64.60**
	2	2752021	1.07	156.71	1.08	1.11	89.13	1.11	1.16	269.19	**1.07**	1.09	35.71	1.15	1.17	591.31	**1.07**	**1.07**	68.15
	3	2737769	**1.10**	191.60	1.09	1.10	126.33	1.22	1.28	210.10	1.13	1.15	130.43	1.29	1.30	591.78	**1.09**	1.13	**70.37**
	4	2748216	1.07	136.87	1.05	1.12	149.59	1.10	1.13	210.73	1.10	1.10	151.61	1.06	1.07	591.67	**1.05**	**1.07**	**67.09**
	5	2702350	**1.25**	145.07	1.24	1.24	54.96	1.31	1.34	188.84	1.26	1.27	74.20	1.29	1.34	592.50	**1.23**	**1.25**	**67.90**
	Average		1.98	162.02	1.96	2.10	78.63	2.03	2.10	245.91	1.95	1.97	71.20	2.00	2.06	449.09	**1.93**	**1.95**	**56.86**

AVG, BST in percentage and Time in seconds.
[a] Fernandes et al. (2014).
[b] Guo et al. (2017).
[c] Biajoli et al. (2019).
[d] Gonzalez et al. (2019).

Table 4
Results obtained for the second set of instances (100 factories, 200 warehouses and 400 customers).

Instances			GA[a]		CS+CPLEX			HEA/FA[b]			BRKGA+LS[c]			GRASPH[d]			CS-ALNS-LB		
Class	ID	LowerB	AVG	Time	BST	AVG	Time	BST	AVG	Time	BST	AVG	Time	BST	AVG	Time	BST	AVG	Time
1	1	1475952	**0.55**	1268.12	0.10	0.30	384.79	0.29	0.33	1390.57	0.10	0.11	**265.79**	**0.10**	**0.10**	**381.21**	**0.09**	**0.11**	339.69
	2	1462736	1.01	1250.09	0.34	0.70	716.06	0.27	0.43	1569.57	**0.12**	0.13	547.62	**0.12**	**0.12**	**130.86**	**0.12**	**0.20**	**231.41**
	3	1492163	0.34	1367.04	0.54	1.00	654.10	0.23	0.48	1535.84	**0.15**	**0.17**	**192.48**	0.15	**0.15**	**281.03**	**0.16**	**0.20**	266.18
	4	1459076	0.49	1285.78	0.24	0.49	740.44	0.25	0.28	1163.24	**0.22**	**0.23**	284.96	**0.22**	0.28	484.13	**0.22**	0.24	**240.00**
	5	1490742	0.67	1303.93	0.11	0.33	**850.42**	0.17	0.24	1555.82	**0.12**	**0.12**	**515.29**	**0.12**	**0.12**	**82.79**	**0.11**	**0.12**	192.28
2	1	970908.5	**0.89**	675.48	**0.26**	0.52	989.39	0.63	0.70	979.27	0.27	**0.28**	398.44	**0.27**	0.36	584.20	**0.26**	**0.30**	**310.07**
	2	965908.5	0.74	662.96	**0.28**	0.46	668.55	0.47	0.56	941.41	0.29	**0.29**	458.38	**0.28**	0.34	559.24	**0.28**	**0.33**	**257.51**
	3	975499.7	1.42	650.19	**0.14**	0.25	**992.58**	0.20	0.25	955.61	**0.14**	**0.15**	**89.55**	**0.14**	**0.14**	487.39	**0.14**	**0.17**	236.91
	4	973019.1	0.56	657.63	**0.28**	0.40	688.28	0.56	0.59	1123.11	**0.28**	**0.29**	594.11	0.35	0.41	592.81	**0.29**	**0.32**	**326.07**
	5	941567	1.12	646.23	0.60	0.65	858.06	**0.86**	0.96	1201.59	**0.61**	**0.61**	**506.97**	**0.86**	1.06	592.22	**0.60**	**0.69**	**283.63**
3	1	5213566	1.63	617.24	**1.62**	**1.63**	**1113.37**	1.91	2.03	886.87	1.64	1.65	704.88	1.79	1.92	**598.74**	**1.62**	**1.66**	600.74
	2	5191321	1.67	601.51	**1.65**	**1.65**	1312.48	1.96	1.99	1264.97	1.68	1.71	**234.85**	1.84	1.94	596.33	1.67	1.71	503.07
	3	5145991	**1.58**	597.43	**1.57**	1.58	**958.88**	1.78	1.84	1236.61	**1.63**	**1.63**	**90.64**	1.77	1.84	594.41	**1.58**	**1.61**	491.88
	4	5225601	1.74	622.04	**1.72**	1.73	1033.46	1.98	2.01	1003.95	**1.74**	**1.75**	**313.32**	2.01	2.09	600.38	**1.72**	**1.76**	498.75
	5	5163182	**1.72**	629.84	**1.67**	**1.69**	1073.08	1.99	2.08	976.64	1.72	1.75	**409.97**	2.02	2.03	594.38	**1.73**	1.75	518.88
4	1	1052172	**0.82**	577.91	**0.61**	0.87	1040.75	0.91	1.04	910.67	**0.64**	**0.69**	437.63	0.73	0.74	594.13	**0.65**	**0.78**	**313.52**
	2	1043553	**0.93**	560.18	0.83	0.88	852.22	0.82	0.89	697.12	0.68	0.74	496.97	0.77	0.85	593.65	**0.67**	**0.71**	**314.25**
	3	1050683	1.88	584.40	**0.62**	0.81	677.12	1.00	1.29	909.24	0.73	**1.04**	566.19	1.20	1.77	592.49	**0.78**	**1.12**	**299.23**
	4	1044571	0.96	592.63	**0.74**	0.94	1099.22	1.36	1.62	826.32	0.81	**0.88**	573.03	0.98	1.01	594.38	**0.80**	**0.90**	**303.95**
	5	1053869	**0.64**	607.94	0.56	0.89	543.26	0.94	1.06	873.75	0.58	**0.61**	494.88	0.56	0.65	594.12	**0.52**	**0.63**	**295.09**
5	1	5486098	0.48	706.40	**0.38**	**0.40**	801.30	**0.61**	0.62	1002.84	**0.39**	0.41	358.97	0.43	0.54	593.59	**0.38**	**0.40**	**326.89**
	2	5461680	0.47	683.08	0.39	0.44	849.41	0.41	0.44	1054.17	**0.41**	0.43	500.54	**0.40**	0.42	593.52	**0.38**	**0.41**	**285.26**
	3	5425391	**0.62**	672.63	0.49	0.53	770.91	0.64	0.73	1421.97	0.42	**0.45**	555.96	0.59	0.59	594.61	**0.39**	0.48	**259.29**
	4	5494811	0.52	689.38	0.43	0.47	657.66	0.58	0.58	1123.96	0.43	0.44	363.07	0.50	0.51	593.16	**0.41**	**0.43**	**322.00**
	5	5442621	**0.47**	670.38	0.39	0.45	1323.85	0.43	0.47	1205.01	0.39	0.40	502.99	0.46	0.48	592.62	**0.38**	**0.40**	**262.75**
Average			0.96	767.22	0.66	0.80	865.98	0.85	0.94	1112.40	0.65	**0.68**	418.30	0.75	0.82	523.85	**0.64**	**0.70**	**331.17**

AVG, BST in percentage and Time in seconds.
[a] Fernandes et al. (2014).
[b] Guo et al. (2017).
[c] Biajoli et al. (2019).
[d] Gonzalez et al. (2019).

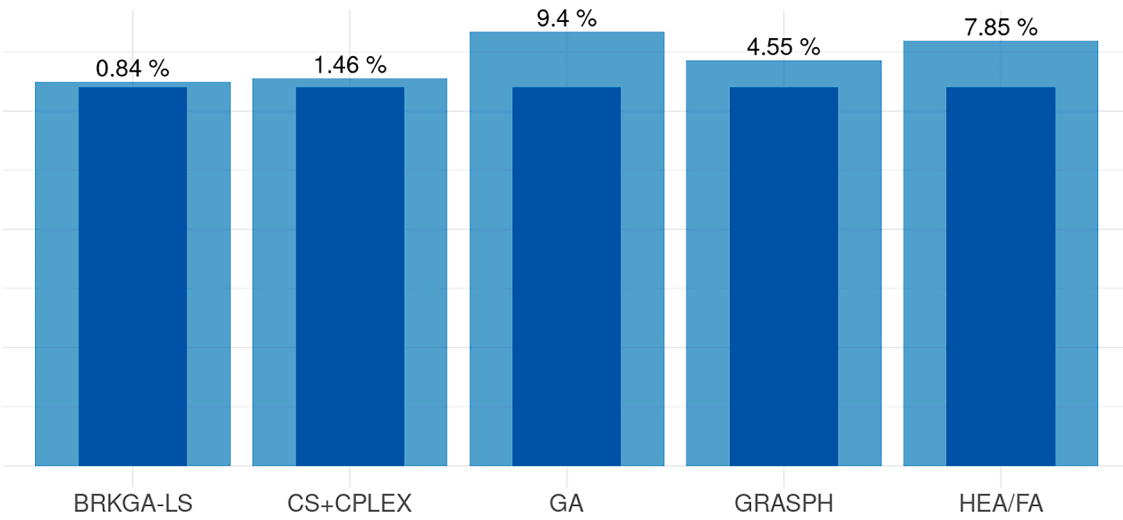

Fig. 7. Comparison of the summation of the best results, presented in the literature, between CS-ALNS-LB and the other methods.

Table 5
Number of times the Null Hypothesis was rejected, in the hypothesis test,when comparing CS-ALNS-LB with CS+CPLEX and BRKGA-LS.

Instance groups	CS-ALNS-LB	
	w/ CS+CPLEX	w/BRKGA-LS
50	19/25	17/25
100	16/25	5/25

- Null Hypothesis (H0): There are no significant difference between CS-ALNS-LB and the other methods
- Alternative Hypothesis (H1): There are significant difference between CS-ALNS-LB and the other methods

Our tests, presented in Table 5, shown the number of instances for which the Null Hypothesis was rejected. For instances with 50 factories, it is clear that the CS-ALNS-LB is different from the other methods since, for more than 60% of the tested instances, the Null Hypothesis was rejected. Considering the results for instances with 100 factories, one may verify that when comparing the CS-ALNS-LB with the CS+CPLEX, for 16 out of 25 instances the Null Hypothesis was rejected, proving that they are different. Now, still in the set of instances with 100 factories, when comparing the CS-ALNS-LB with BRKGA-LS, the Null Hypothesis is rejected just for 5 out of 25 instances, which may lead us to conclude that there is no significant difference between both algorithms for this set of instances. This could have happened thanks to the fact that the mean of averages solution of the BRKGA-LS is lower than the one obtained by CS-ALNS-LB.

5. Conclusions and future works

This work presented a hybridization of CS and ALNS metaheuristics, which to the best of our knowledge, has not been addressed in the literature. In addition, CS-ALNS was also combined with a Local Branching technique, resulting in the proposed CS-ALNS-LB matheuristic.

The performance of the proposed method was compared with the most relevant approaches in the literature. The obtained results showed that the CS-ALNS-LB outperformed the other approaches both in the quality of the solution found, as well as in the computational time spent. Regarding the quality of the solutions found, for the instances with 50 factories, CS-ALNS-LB found mean results for both best and average solutions better than all the other analyzed algorithms. Besides, in terms of computational time, CS-ALNS-LB was, on average, 1.20 times faster than state-of-the-art methods (CS-CPLEX and BRKGA-LS). For the instances with 100 factories, despite CS-ALNS-LB having lost performance, it still obtained mean best solution better than those from all analyzed algorithms, while the mean average solution was worse just for BRKGA-LS. In terms of computational time, for the instances of 100 factories, CS-ALNS-LB was, on average, 1.20 times faster than state-of-the-art methods (CS-CPLEX and BRKGA-LS). Also, one should not forget that BRKGA-LS is a parallel metaheuristic, so the reduction in computational time is even greater if one consider it a sequential approach. In addition to the numerical results, a statistical analysis was presented, showing that the CS-ALNS-LB differs from other literature approaches not only by its randomness.

As future work, a natural path to follow would be to work on cutting plane algorithms to improve the performance of Local Branching. Also, destructive and repairer procedures can be added in order to better explore the solution space.

CRediT authorship contribution statement

Gabriel Souto: Conceptualization, Methodology, Software, Writing - original draft, Writing - review & editing. **Igor Morais:** Conceptualization, Methodology, Software, Formal analysis, Investigation, Writing - original draft, Writing - review & editing. **Geraldo Regis Mauri:** Writing - original draft, Writing - review & editing. **Glaydston Mattos Ribeiro:** Writing - review & editing. **Pedro Henrique González:** Conceptualization, Methodology, Software, Formal analysis, Investigation, Writing - original draft, Writing - review & editing, Supervision, Project administration.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This research was partially supported by Conselho Nacional de Desenvolvimento Científico e Tecnológico - CNPq (processes 307835/2017-0 and 302261/2019-2), Fundação Carlos Chagas Filho de Amparo à Pesquisa do Estado do Rio de Janeiro - FAPERJ (Process 233926) and Coordenação de Aperfeiçoamento de Pessoal de Nível Superior.

References

Ahuja, R. K., Magnanti, T. L., & Orlin, J. B. (1988). *Network flows*.

Biajoli, F. L., Chaves, A. A., & Lorena, L. A. N. (2019). A biased random-key genetic algorithm for the two-stage capacitated facility location problem. *Expert Systems With Applications, 115*, 418–426.

Chaves, A., Lorena, L., Senne, E., & Resende, M. (2016). Hybrid method with cs and brkga applied to the minimization of tool switches problem. *Computers & Operations Research, 67*, 174–183. https://doi.org/10.1016/j.cor.2015.10.009

de Alvarenga Rosa, R., Machado, A. M., Ribeiro, G. M., & Mauri, G. R. (2016). A mathematical model and a clustering search metaheuristic for planning the helicopter transportation of employees to the production platforms of oil and gas. *Computers & Industrial Engineering, 101*, 303–312.

Fernandes, D. R. M., Rocha, C., Aloise, D., Ribeiro, G. M., Santos, E. M., & Silva, A. (2014). A simple and effective genetic algorithm for the two-stage capacitated facility location problem. *Computers & Industrial Engineering, 75*, 200–208.

Fischetti, M., & Lodi, A. (2003). Local branching. *Mathematical Programming, 98*, 23–47.

Geoffrion, A. M., & Graves, G. W. (1974). Multicommodity distribution system design by benders decomposition. *Management Science, 20*, 822–844.

Gonzalez, P. H., & Brandão, J. (2018). A biased random key genetic algorithm to solve the transmission expansion planning problem with re-design. In *2018 IEEE congress on evolutionary computation (CEC)* (pp. 1–7). IEEE.

González, P. H., Clímaco, G., Mauri, G. R., Vieira, B. S., Ribeiro, G. M., Orrico Filho, R. D., Simonetti, L., Perim, L. R., & Hoffmann, I. C. S. (2019). New approaches for the traffic counting location problem. *Expert Systems With Applications, 132*, 189–198.

Gonzalez, P. H., Gabriel, S., et al. (2019). Grasp híbrido para resolução do problema de localização de facilidades capacitadas em dois níveis. Simpósio Brasileiro de Pesquisa Operacional - SBPO (pp. 1–11).

Gonzalez, P. H., Simonetti, L., Michelon, P., Martinhon, C., & Santos, E. (2016). A variable fixing heuristic with local branching for the fixed charge uncapacitated network design problem with user-optimal flow. *Computers & Operations Research, 76*, 134–146.

Guo, P., Cheng, W., & Wang, Y. (2017). Hybrid evolutionary algorithm with extreme machine learning fitness function evaluation for two-stage capacitated facility location problems. *Expert Systems With Applications, 71*, 57–68.

Hettmansperger, T. P., & McKean, J. W. (1998). *Robust nonparametric statistical methods*.

Keskin, M., & Çatay, B. (2018). A matheuristic method for the electric vehicle routing problem with time windows and fast chargers. *Computers & Operations Research, 100*, 172–188. https://doi.org/10.1016/j.cor.2018.06.019

Kiefer, A., Hartl, R. F., & Schnell, A. (2017). Adaptive large neighborhood search for the curriculum-based course timetabling problem. *Annals of Operations Research, 252*, 255–282.

Klose, A. (2000). A lagrangian relax-and-cut approach for the two-stage capacitated. *European Journal of Operational Research, 126*, 185–198.

Klose, A., & Drexl, A. (2005). Facility location models for distribution system design. *European Journal of Operational Research, 162*, 4–29.

Laporte, G., Musmano, R., & Vocaturo, F. (2010). An adaptive large neighborhood search heuristic for the capacitated arc-routing problem with stochastic demands. *Transportation Science, 44*, 125–135.

Litvinchev, I., & Ozuna, E. L. (2012). Lagrangian bounds and a heuristic for the two-stage capacitated facility location problem. *International Journal of Energy Optimization and Engineering, 1*, 59–71.

López-Ibáñez, M., Dubois-Lacoste, J., Cáceres, L. P., Birattari, M., & Stützle, T. (2016). The irace package: Iterated racing for automatic algorithm configuration. *Operations Research Perspectives, 3*, 43–58.

Louzada, R. R., Mauri, G. R., & Ribeiro, G. M. (2016). Método heurístico híbrido para resolução do problema de localização de facilidades capacitadas em dois níveis. In *Simpósio Brasileiro de Pesquisa Operacional - SBPO* (pp. 2460–2471).

Mauri, G. R., Biajoli, F. L., Rabello, R. L., Chaves, A. A., Ribeiro, G. M., & Lorena, L. A. N. (2021). Hybrid metaheuristics to solve a multiproduct two-stage capacitated facility location problem. *International Transactions in Operational Research*. n/a.

Mauri, G. R., Ribeiro, G. M., Lorena, L. A. N., & Laporte, G. (2016). An adaptive large neighborhood search for the discrete and continuous berth allocation problem. *Computers & Operations Research, 70*, 140–154.

Oliveira, A. C. A. M. d., Chaves, A. A., & Lorena, L. A. N. (2013). Clustering search. *Pesquisa Operacional, 33*, 105–121.

Oliveira, A. C., & Lorena, L. A. (2007). Hybrid evolutionary algorithms and clustering search. In *HYBRID evolutionary algorithms* (pp. 77–99). Springer.

Penna, P. H. V., Subramanian, A., & Ochi, L. S. (2013). An iterated local search heuristic for the heterogeneous fleet vehicle routing problem. *Journal of Heuristics, 19*, 201–232.

Pirkul, H., & Jayaraman, V. (1998). A multi-commodity, multi-plant, capacitated facility location problem: formulation and efficient heuristic solution. *Computers & Operations Research, 25*, 869–878.

Ribeiro, G. M., Laporte, G., & Mauri, G. R. (2012). A comparison of three metaheuristics for the workover rig routing problem. *European Journal of Operational Research, 220*, 28–36.

Ropke, S., & Pisinger, D. (2006). An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. *Transportation Science, 40*, 455–472.

Tragantalerngsak, S., Holt, J., & Ro"nnqvist, M. (1997). Lagrangian heuristics for the two-echelon, single-source, capacitated facility location problem. *European Journal of Operational Research, 102*, 611–625.

Tragantalerngsak, S., Holt, J., & Ronnqvist, M. (2000). An exact method for the two-echelon, single-source, capacitated facility location problem. *European Journal of Operational Research, 123*, 473–489.

Wu, T., Yang, Z., Chu, F., & Zhou, Z. (2015). A lagrangean relaxation approach for a two-stage capacitated facility location problem with choice of depot size. In *2015 IEEE 12th international conference on networking, sensing and control* (pp. 39–44). IEEE.

Yang, Z., Chen, H., Chu, F., & Wang, N. (2019). An effective hybrid approach to the two-stage capacitated facility location problem. *European Journal of Operational Research, 275*, 467–480.

Zhong, R. Y., Xu, X., Klotz, E., & Newman, S. T. (2017). Intelligent manufacturing in the context of industry 4.0: A review. *Engineering, 3*, 616–630.